Sử dụng Git & GitHub

Git là một trong các hệ thống quản lý phiên bản phân tán phổ biến nhất hiện nay. Cài đặt Git:

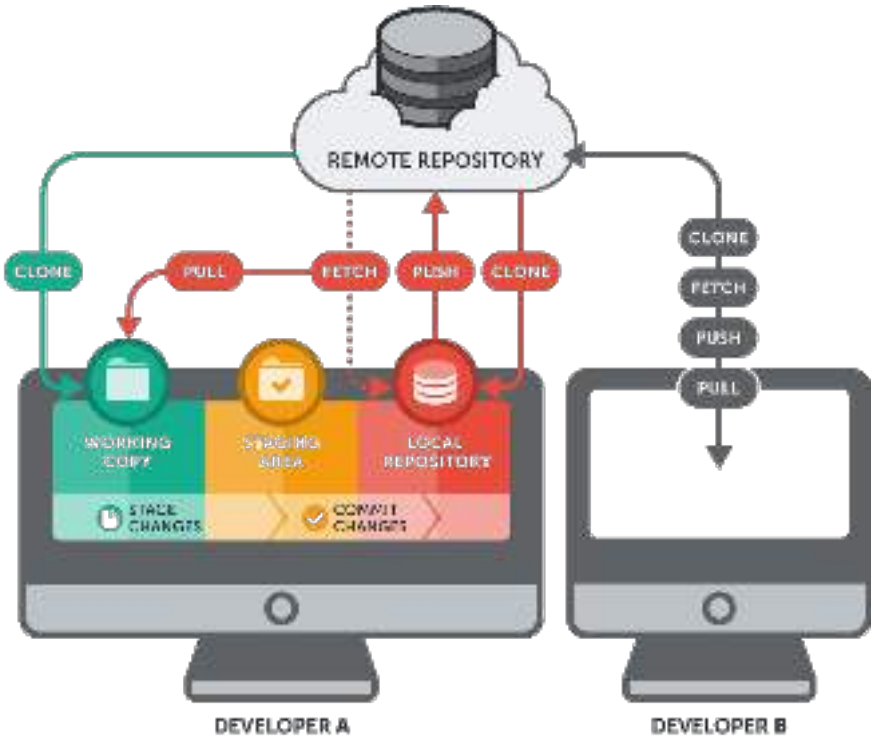
- Tải Git tại https://git-for-windows.GitHub.io/. Double-click vào file cài đặt và thực hiện theo hướng dẫn.
- Để kiểm tra cài đặt thành công, mở Command Prompt gõ lệnh "git --version" kết quả sẽ hiển thị giống như sau:

```
Command Prompt
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\dhthanh>git --version
git version 2.10.2.windows.1
```

- Tại Command Prompt thiết lập username và email cho git.

```
C:\Users\dhthanh>git config --global user.name "thanh duong"

C:\Users\dhthanh>git config --global user.email "dhthanhqa@gmail.com"
```

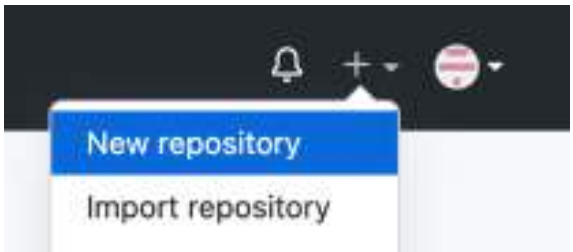
GitHub là một nền tảng cung cấp kho lưu trữ mã nguồn git. Nó cho phép nhiều người làm việc trên một project ở bất cứ nơi đâu. Địa chỉ GitHub: http://github.com/. Bên dưới các thao tác cơ bản làm việc git & github.



<u>Các thao tác trên Github</u>

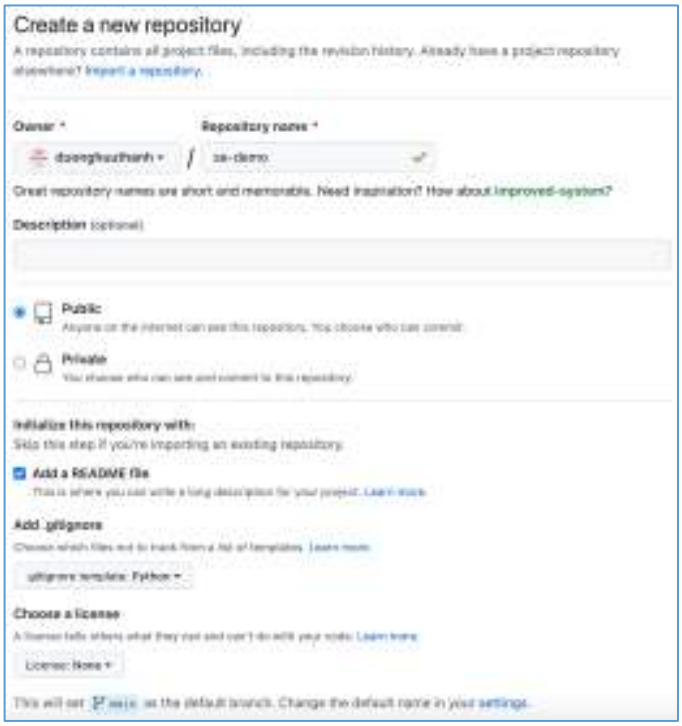
Tạo thùng chứa

Sau khi đăng nhập, ở gốc trên phải click biểu tượng dấu "+" chọn "New repository"

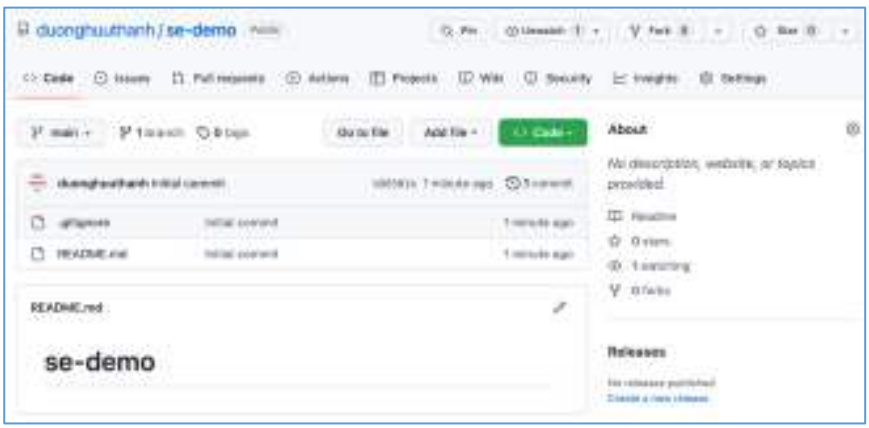


Một số thông tin cần chú ý khi tạo thùng chứa

- Tên thùng chứa thường là tên project và không trùng tên với các thùng chứa đã có.
- Chọn chính sách cho thùng chứa: nếu là Public thì mọi người đều có thể thấy thùng chứa, nếu là Private thì chỉ mình và thành viên mình thêm vào mới có thể thấy thùng chứa. Trong hai lựa chọn, ta đều có thể chỉ định người được commit thùng chứa.
- Ta có thể tích chọn "Add a README file" để tạo tập tin Readme chứa các mô tả chi tiết của thùng chứa. Mặc dù yêu cầu này là tuỳ chọn, nhưng ta nên luôn tạo ra tập tin này.
- Tạo tập tin .gitignore cấu hình những thành phần trong thùng chưa mà git không nên quan tâm, trong select box của nó ta có thể chọn ngôn ngữ thực hiện thùng chứa để Github tạo ra một số cấu hình có sẵn.



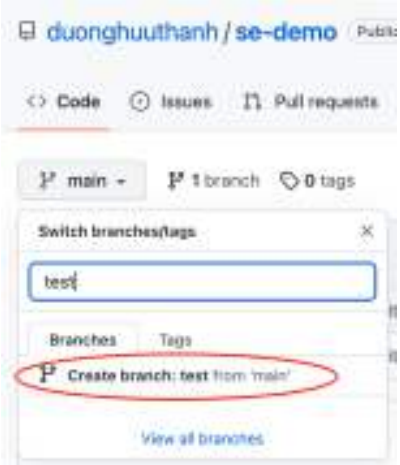
Hình Error! No text of specified style in document.**.1. Tạo thùng chứa Github (truy cập 03/01/2023)**



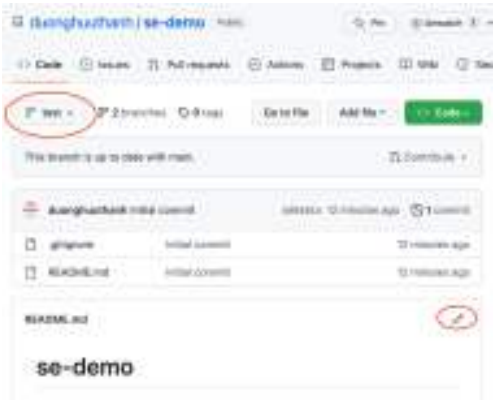
Tạo nhánh

Nhánh là cách thức làm việc trên một phiên bản khác của thùng chứa tại cùng thời điểm. Mặc định khi vừa tạo thùng chứa, GitHub tạo sẵn nhánh main (ta có thể đổi tên mặc định lúc tạo thùng chứa). Ví dụ tạo nhánh test từ nhánh main

- Tại thùng chứa, click vào select box “main”
- Gõ tên nhánh (develop) muốn tạo ở và click “Create branch: test”



Ta thực hiện một số thay đổi trong nhánh test. Đầu tiên chọn nhánh test và chỉnh sửa tập tin README.



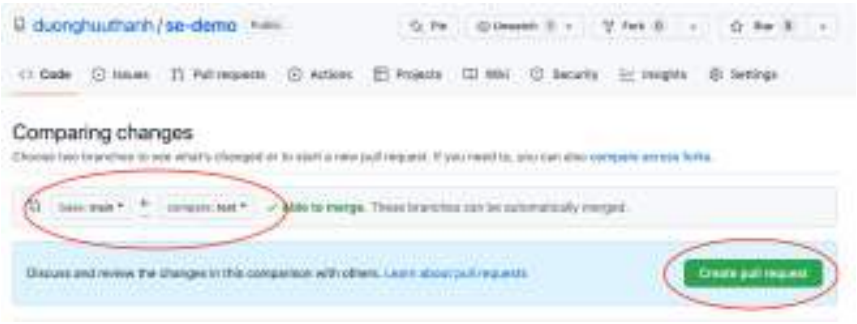
Chỉnh sửa xong tập tin README, ta thực hiện Commit changes.

Tạo Pull Request

Pull Request (PR) dùng để đề nghị (proposing) những thay đổi và yêu cầu để ai đó xét duyệt (review) trộn (merge) từ một nhánh này vào nhánh nào khác. PR sẽ thể hiện sự khác nhau nội dung giữa hai nhánh

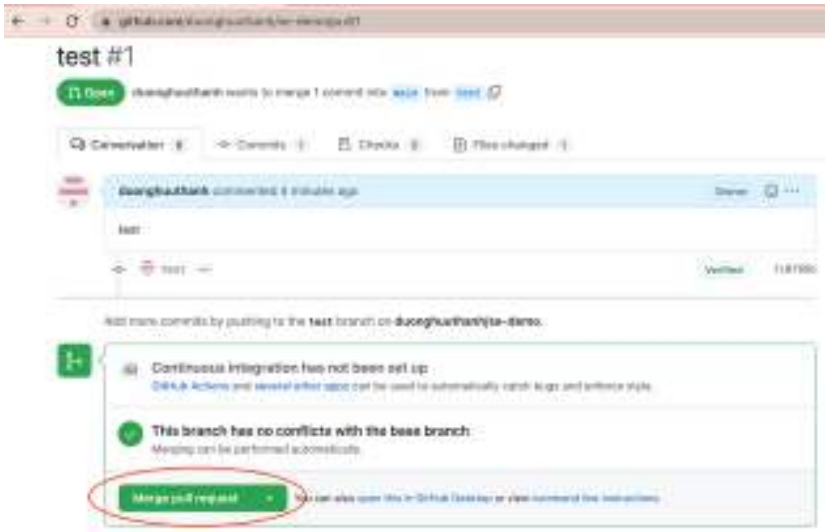
Tại thùng chứa, click chọn tab “Pull Requests” và click vào button “New pull request”

Để yêu cầu trộn nhánh test vào nhánh main, ta chọn như bên dưới và click “Create pull request”



Trộn Pull Request

Sau khi tạo PR, các thay đổi được tập trung trong PR để có thể xét duyệt và quyết định trộn hay không. Để merge PR, tại PR vừa tạo click “Merge pull request” và “Confirm merge”. Sau đó quay lại branch main, ta sẽ thấy nhưng thay đổi trên test được áp dụng vào nhánh main.



Thêm thành viên vào thùng chứa

Để các thành viên có thể push code vào thùng chứa cần thêm họ thành cộng tác (collaborator) vào thùng chứa. Các bước thực hiện

- Tại thùng chứa, click tab Settings, và click vào mục Collaborators ở mục menu bên trái.
- Click “Add people” và nhập tên thành viên muốn mời, lời mời sẽ được gửi vào mail thành viên và chờ họ chấp nhận/từ chối.

<u>Các thao tác cơ bản trên git</u>

- git init: khởi tạo thùng chứa cục bộ

- git clone: lấy thùng chứa từ remote về local.
- git add: thêm những thay đổi từ working copy sang staging area
- git commit: thêm thay đổi từ staging area đến local repository
- git push: đưa thay đổi từ local repository lên remote repository (github)
- git stash: lưu tạm các thay đổi nhánh hiện tại vào stash
- git diff: so sánh sự khác nhau trang thái hiện tại thùng chứa với trạng thái ban đầu, hoặc giữa hai commit.
- git status: xem trạng thái hiện tại các tập tin trong thùng chứa.
- git merge: trộn thay đổi hai nhánh.
- git log: xem lịch sử commit
- git branch: tạo nhánh hoặc xem danh sách nhánh
- git checkout branch-name: chuyển sang làm việc trên 1 nhánh
- git checkout file-name: phục hồi thay đổi trên tập tin về trạng thái ban đầu.
- git reset file-name: đưa 1 tập tin từ staging area trở về working copy.
- git pull: cập nhật những thay đổi thùng chứa remote về thùng chứa local.
- git fetch: lấy tất cả các nhánh của thùng chứa remote về thùng chức local.
- git rm file-name: xoá tập tin khỏi project và đưa vào staging area để chuẩn bị commit.

Thực hành 1

- Tạo thùng chứa trên github
- Lấy thùng chứa về máy cục bộ (git clone)
- Tạo một số thay đổi tuỳ ý trong thùng chứa và thực hiện quá trình đưa lên thay đổi lên github. (git add → git commit → git push)
- Hai thành viên ngồi cạnh, thêm thành viên kế bên vào thùng chứa, yêu cầu bạn lấy thực hiện push thay đổi lên thùng chưa của mình.

Thực hành 2

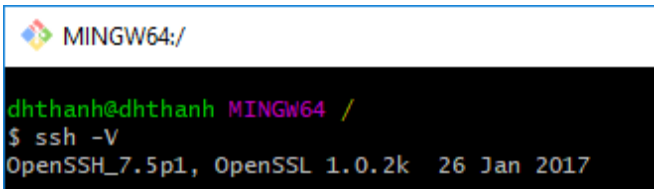
- Dùng lệnh git tạo nhánh (test) trên thùng chứa cục bộ
- Tạo một số thay đổi trên nhánh và push nhánh lên github
- Tiến hành tạo PR để trộn nhánh test vào nhánh main.

- Tạo conflict: sửa cùng một chỗ trên hai nhánh main và test rồi tạo pull request, hoặc hai thành viên sửa cùng một chỗ trên nhánh main và push lên github. Tiến hành giải quyết xung độ mã nguồn và push code lên github.

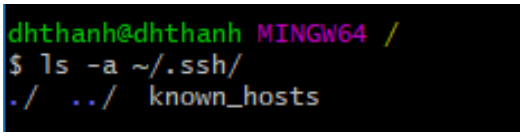
Thực hành 3 **(Cầu hình SSH)**

SSH (Secure Shell) là giao thức dùng để thiết lập kết nối bảo mật, khi sử dụng trong git ta không cần nhập mật khẩu mỗi lần push code. Để sử dụng SSH cần 2 key là public key và private key, trong đó private key đặt máy tính local tại thư mục ~/.ssh và public key sẽ đặt trên git server.

SSH Client được tích hợp sẵn trong Git Bash Shell. Đầu tiên bật Git Bash kiểm tra SSH đã được cài đăt hay chưa: ssh -v



Xem tính trạng thư mục .ssh



Tính trạng trên cho biết chứa có cặp khóa public key và private key, nếu 2 khóa này có thì trong thư mục trên sẽ xuất hiện những tập tin tên id_rsa, id_rsa.pub.

Tại cửa sổ Git Bash gõ lệnh ssh-keygen. Trong quá trình tạo sẽ yêu cầu nhập tên file và passphrase, bấm enter để dùy mặc định.

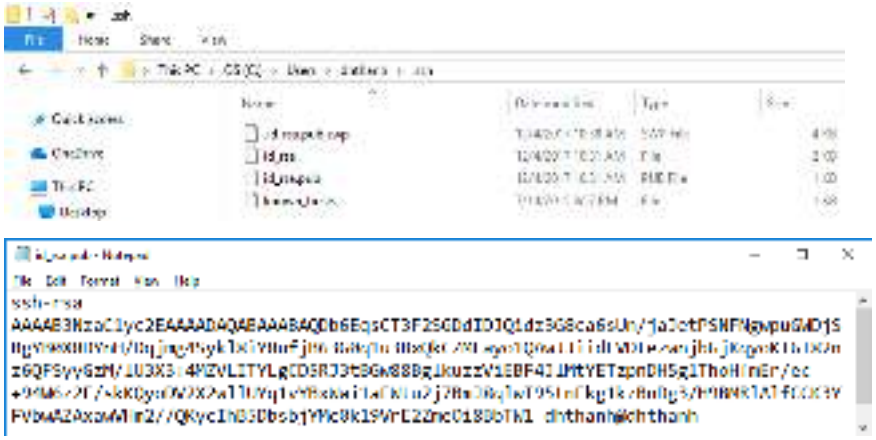
Xem lại tình trạng thư mục .ssh

```
dhthanh@dhthanh MINGW64 /
$ ls -a ~/.ssh/
./  ../  id_rsa  id_rsa.pub  known_hosts
```

Trong đó

- id_rsa là private key
- is_rsa.pub là public key

Đường dẫn tuyệt đối của thư mục .ssh trên Windows là C:\Users\<username>\.ssh, trong ví dụ này là C:\Users\dhthanh\.ssh. Vào thư mục này và mở tập tin id_rsa.pub và sao chép toàn bộ nội dung tập tin.



Đặt public key lên github

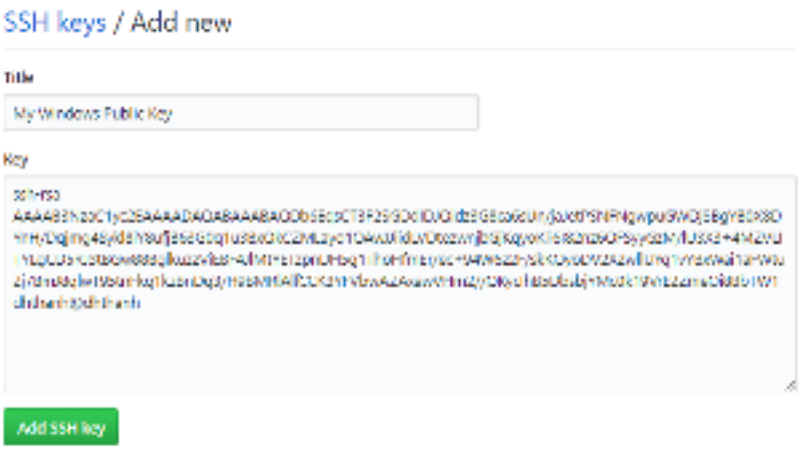
Truy cập vào github và đăng nhập tài khoản cần thiết lập public key.

Tại icon user ở gốc trên bên phải, click sổ menu và chọn Settings.

Click chọn tab “SSH And GPG Keys”

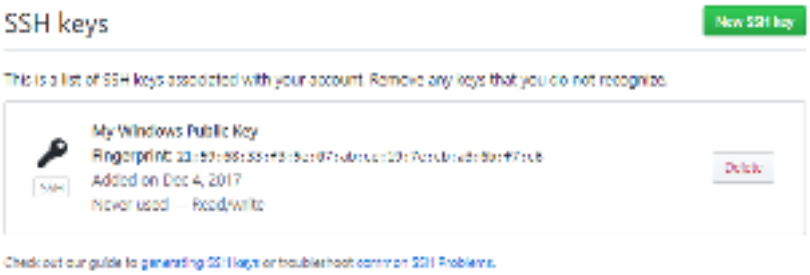
Click button “New SSH Key” để thêm public key mới vào GitHub.

Đặt tên cho public key và sao chéo toàn bộ nội dung trong id_rsa.pub ở local vào phần Key, sau đó bấm “Add SSH key”.



GitHub sẽ yêu cầu nhập lại mật khẩu để xác nhận.

Kết quả public key được thêm vào GitHub



Bây giờ dùng SSH để push code từ local lên GitHub, đầu tiên tại cửa sổ thùng chứa chép git url dùng SSH.

Ta có thể sử dụng url này để clone, push code mà không cần nhập mật khẩu xác thực nữa.

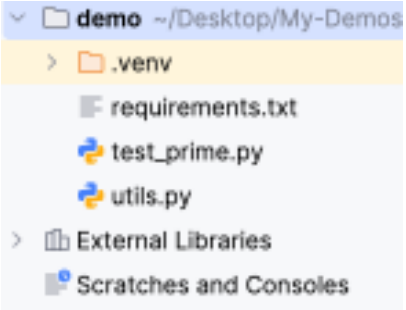
Thực hành 4

Sử dụng Github Issues để quản lý lỗi. Cấu trúc một lỗi khi report:

- Chủ đề lỗi, ví dụ: Đăng nhập thật bại khi mật khẩu lớn hơn 20 ký tự.
- Các bước tái hiện lỗi (reproduce)
- Kết quả mong muốn (Expected result)
- Kết quả thực sự (Actual result)
- Môi trường kiểm thử (Environment)

Thực hành 5

Giả sử, ta tạo thùng chứa đặt tên là TestingDemo. Bên trong thùng chứa này, ta tạo project có cấu trúc giống như sau:



Ta cài thư viện để thực hiện viết unit test:

```
pip install pytest pytest-timeout
```

Xuất danh sách thư viện ra tập tin requirements.txt:

```
pip freeze > requirements.txt
```

Trong tập tin uitls.py chứa hàm kiểm tra số nguyên tố:

```
def is_prime(n):
    if n < 2:
        raise ValueError('Invalid Number')

    for i in range(2, math.isqrt(n) + 1):
        if n % i == 0:
            return False

    return True
```

Trong tập tin test_prime.py (chú ý trong tên có test_ để pytest tự động tìm thực thi unit test) chứa các unit test để kiểm thử hàm is_prime:

```
import pytest
from utils import is_prime

def test_true():
    assert is_prime(2) == True

def test_false():
```

```
    assert is_prime(4) == False

def test_exception():
    with pytest.raises(ValueError):
        is_prime(1)

@pytest.mark.timeout(2)
def test_timeout():
    is_prime(99999)
```

Bây giờ, ta tiến hành các bước để chạy test case tự động Github Actions:

- Trong thư mục thùng chứa, ta tạo thư mục như sau: .github/workflows/
- Tạo tập tin test.yml trong thư mục workflows như sau:

```
name: Run Tests

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.10'

    - name: Install dependencies
      run: pip install -r demo/requirements.txt

    - name: Run tests
      run: PYTHONPATH=demo pytest demo
```

Các khai báo quan trọng trong tập tin:

- name: Run Tests -> tên của workflow hiển thị trong tab Actions.
- on: [push] -> workflow chạy mỗi lần push code.
- jobs: Workflow gồm nhiều job. Mỗi job = 1 máy ảo riêng.
- runs-on: ubuntu-latest -> GitHub tạo một máy ảo Linux để chạy test.
- Steps: Các bước thực thi trong job, giống như script shell chạy tuần tự.

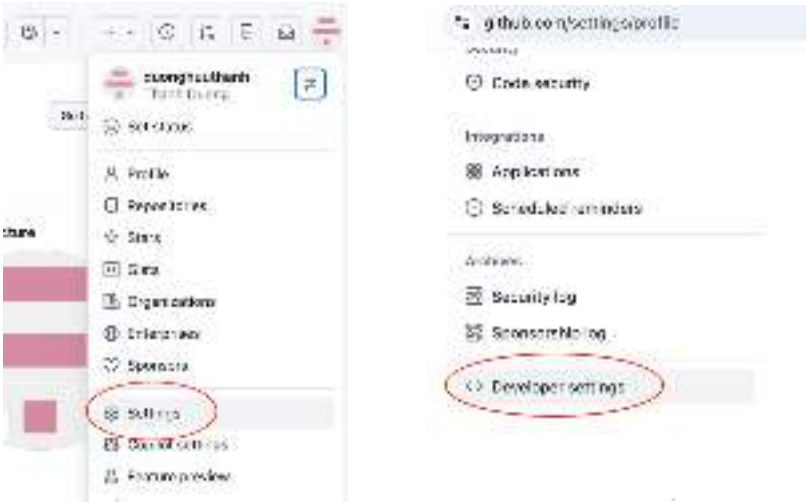
  - uses: actions/checkout@v4: clone mã nguồn từ Github về máy ảo.

- uses: actions/setup-python@v5: cài python trên máy ảo.
- Cài các thư viện và run test.

Sau đó, ta tiến hành push các mã nguồn thay đổi lên github.

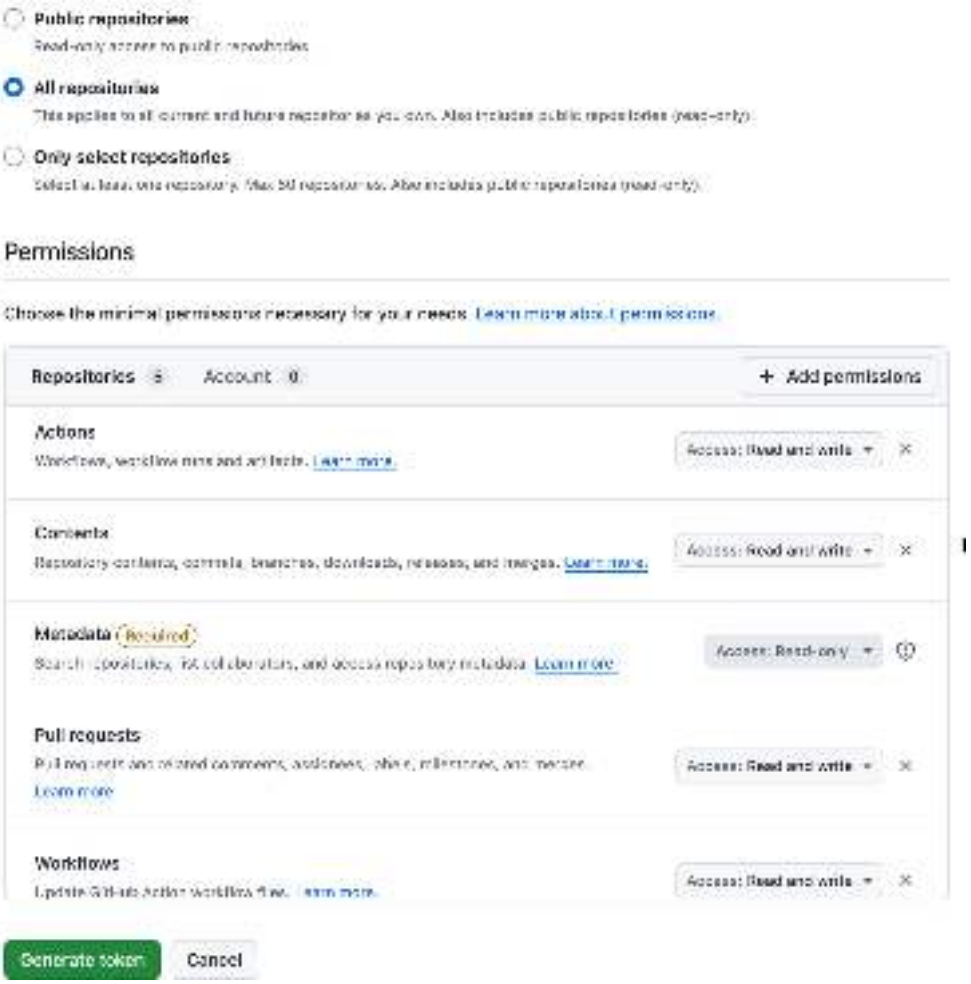
Chú ý trong quá trình push project có thể xuất hiện một số lỗi do quyền của Personal Access Token (PAT). Khi đó, ta có thể xử lý như sau:

Truy cập GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) để tạo mới Token:



Ta có thể thiết lập các quyền sau đây khi tạo token:

- Contents: Read and Write
- Pull requests: Read and Write
- Metadata: Read
- Actions: Read and Write
- Workflows: Read and Write



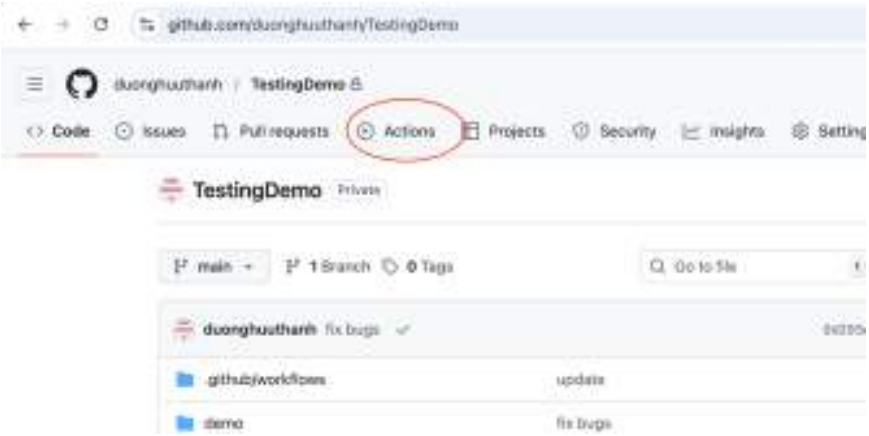
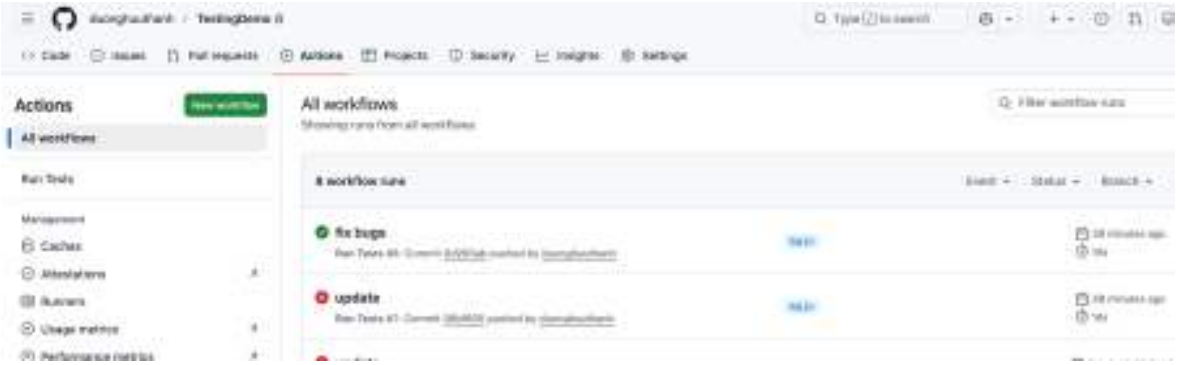
Sau đó, ta tiến hành xoá token trên máy local:

- Trên windows dùng lệnh: git credential-manager clear
- Trên macos dùng lệnh: git credential-osxkeychain erase

Sau đó, push project lại thì git sẽ hỏi:

- Username: your_github_username
- Password: <dán PAT mới vào đây>

Sau khi push thành công, truy cập lại thùng chứa Github và vào mục Actions:



Ta thấy các workflows đang chạy: